Shark Attacks Collector

Projet Final d'Architecture BD, collecte de données, ESEO 2026 - 2027 - Noah GOGEON

Le besoin et le périmètre

Je cherche à répondre à cette question : où et quand les incidents de requins sont-ils recensés, et quelles espèces sont candidates là où GBIF documente une présence ? J'ai conçu le projet pour aider un étudiant, un enseignant ou un analyste à choisir des zones et périodes à étudier. Je ne calcule pas un risque individuel et je ne prétends pas identifier le requin responsable.

Trois usages, trois indicateurs

Usage	Indicateur	Données
Repérer les lieux à examiner	Incidents déclarés par pays, localité et année	GSAF curated
Comparer les mois	Incidents dont la date est renseignée par mois	Dates GSAF
Orienter une recherche écologique	Proximités aux occurrences à 25 et 100 km	Attaques géocodées + GBIF

Source et choix de collecte

J'ai retenu l'export CSV du Global Shark Attack File publié sur Kaggle. La collecte est ponctuelle; je ne scrape pas le site et ne contourne pas son authentification. L'export contient des noms, donc je ne le verse pas dans le dépôt public sans autorisation de redistribution. Pour lancer l'analyse sur les données réelles, je télécharge le CSV depuis le lien ci-dessous, je le décompresse si nécessaire et le renomme `global_shark_attacks_raw.csv`. Je le place ensuite dans le dossier `data/raw/` du projet. Depuis la racine du dépôt, après l'installation décrite dans le README, je lance `python -m src.pipeline`. Sans export réel local, le pipeline utilise l'échantillon synthétique de 121 lignes fourni dans le dépôt.

Télécharger le CSV depuis le projet Kaggle

Je conserve l'export réel localement plutôt que de le versionner : il peut contenir des données identifiantes et reste soumis aux conditions du producteur. Le CSV synthétique permet aux autres personnes de cloner et tester le pipeline sans récupérer ces données.

Décision	Raison
Batch ponctuel	L'export source n'est pas un flux continu et sa mise à jour n'est pas garantie.
CSV locaux	À 6 890 lignes, les fichiers sont faciles à inspecter et évitent une base sans besoin mesurable.
Snapshot brut	Une copie UTC non transformée conserve la preuve et permet le rejeu.

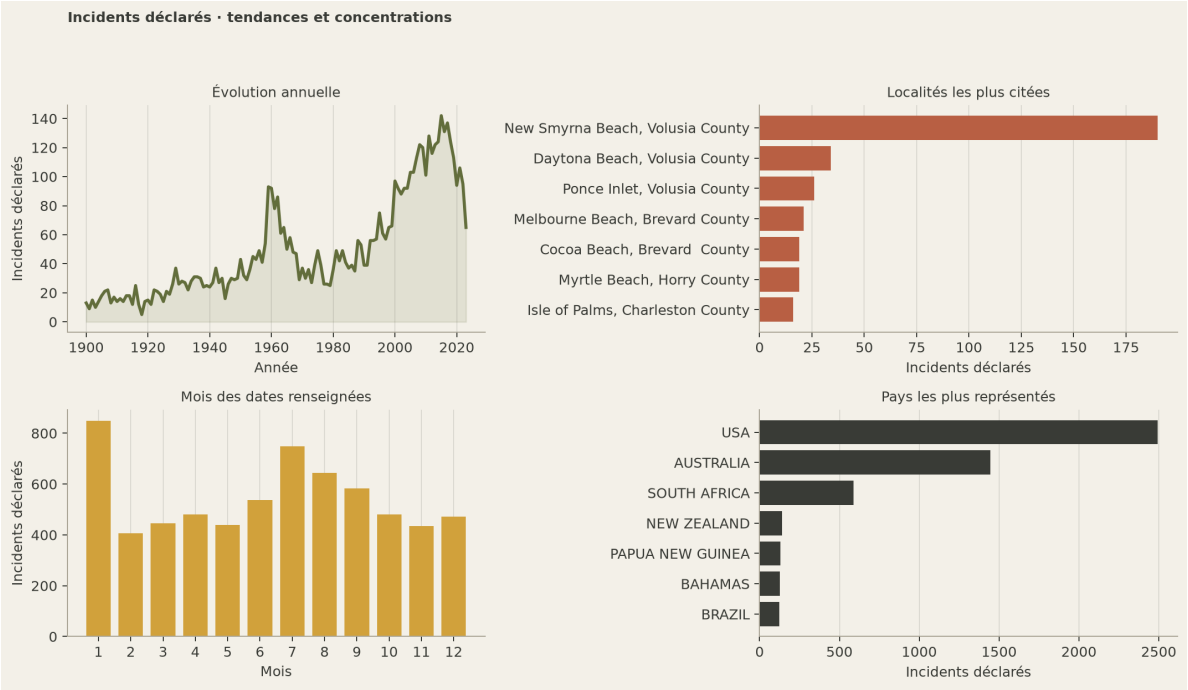
J'ai défini le grain comme un incident déclaré. Comme la source ne fournit pas de clé naturelle, je contrôle l'unicité sur tous les champs disponibles. La règle est reproductible, mais pourrait fusionner deux événements distincts dont les valeurs seraient identiques.

Sources complémentaires et limites du sujet

Source	Apport au projet	Précaution
GSAF via Kaggle	Source principale des incidents déclarés	Export ponctuel; droits de redistribution à vérifier
GBIF	Occurrences taxonomiques et géographiques	Présence observée, non abondance ni identification d'une attaque
Nominatim / OSM	Coordonnées approximatives des lieux textuels	Centre de ville possible; pas une position GPS d'incident

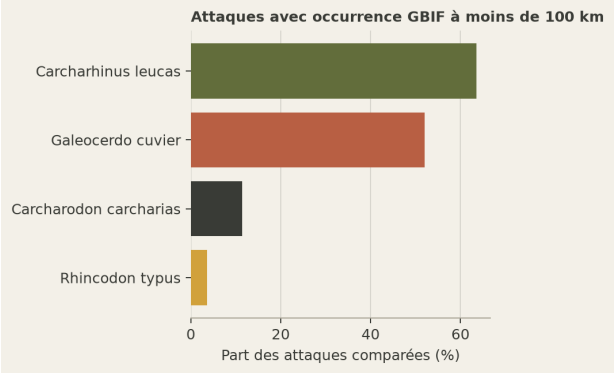
Ce que montrent les données

Dashboard reconstruit depuis les lignes acceptées



Dans les données que j'ai traitées, les signalements augmentent fortement et culminent autour des années 2010. Janvier est le mois le plus représenté parmi les dates renseignées. New Smyrna Beach arrive nettement en tête des localités connues. J'interprète ces volumes comme le contenu du fichier : ils peuvent aussi refléter la fréquentation, la couverture géographique ou les pratiques de déclaration.

Rapprochement exploratoire avec GBIF



J'ai classé les espèces selon les occurrences GBIF géographiquement proches. Une attaque peut être proche de plusieurs espèces : les catégories ne sont pas exclusives et ne désignent pas l'espèce responsable.

J'observe un maximum dans les années 2010. La baisse récente peut correspondre à une période incomplète ou à un changement de déclaration; elle ne prouve pas une baisse des événements réels.

J'utilise les attaques comparées comme dénominateur. Une même attaque peut être proche de plusieurs espèces : le graphique ne permet pas une attribution exclusive.

Comment interpréter cette proximité

J'ai comparé 1 011 lieux d'attaque géocodés aux occurrences GBIF disponibles. Les coordonnées d'attaque sont parfois des centres de localité et les occurrences dépendent de l'effort d'observation. J'interprète la proximité comme une présence documentée dans la région, pas comme celle d'un animal à la date de l'incident. Une occurrence juvénile ne prouve pas une nurserie.

Traçabilité et fiabilité des données

Une copie brute qui permet de rejouer



À chaque exécution, je conserve une copie non transformée du CSV dans un snapshot horodaté UTC. Mon rapport garde son nom, son empreinte SHA-256 et sa taille. Je profile la source avant validation pour relever ses dimensions, types, valeurs manquantes, doublons et modalités fréquentes. Cette trace me permet de comparer l'entrée au résultat sans réécrire la source.

Cinq contrôles, deux chemins

Contrôle	Condition	Traitement
Complétude	Pays présent	Rejet avec motif
Validité	Année plausible	Rejet avec compteur
Cohérence	Type reconnu	Rejet avec motif
Domaine	Âge entre 0 et 120	Âge corrigé à NULL
Unicité	Ligne source non répétée	Doublon supprimé et compté

Je transforme les lignes acceptées en curated et conserve séparément les lignes rejetées avec leur cause. Je remplace ces deux fichiers à chaque run : relancer le traitement ne cumule pas de doublons dans les sorties.

Échec et reprise

J'ai testé une source absente : mon pipeline écrit un rapport FAIL avec le run_id, le type et le message d'erreur, puis retourne un code d'échec. Après correction de la source, je relance le même point d'entrée; un nouveau run est créé. Le snapshot déjà collecté reste disponible pour diagnostic.

Choix de stockage

J'utilise raw comme zone de rejeu, et les CSV curated et rapports pour l'analyse. Je n'ai pas ajouté de Warehouse ou de Mart séparé, car je n'ai pas identifié de bénéfice pour un export ponctuel de cette taille. Je reconsidérerai SQLite si des requêtes répétées ou plusieurs sources intégrées deviennent un besoin concret.

Architecture, qualité et reprise

Trajet d'une ligne



La V2 ajoute le profilage, le contrôle de schéma, la séparation accepted/rejected et l'observabilité. Le choix de garder un batch CSV, plutôt que d'introduire SQLite ou du streaming, est détaillé dans l'ADR 001; les deux vues d'architecture sont livrées en SVG.

Contrôles et compteurs du dernier run réel

Run	Entrées	Acceptées	Rejetées	Doublons
PASS	6890	6687	194	9

Le traitement s'est terminé normalement : **PASS**. Le contrôle qualité reste **PASS_WITH_WARNINGS** parce que 194 lignes ont été rejetées. Ce n'est pas un échec du lancement : le rapport détaille chaque règle, conserve les rejets et permet de les examiner sans masquer les anomalies.

Contrôle	Échecs / action
Pays	51 · rejet
Année	133 · rejet
Type	20 · rejet
Âge	0 · corrigé à NULL
Unicité	9 · doublons supprimés

Observabilité et reprise

Run 20261018T122103101173Z · durée 0.27 s · dernière date d'incident 2023-08-16 · âge du dernier incident 1159 jours. La fraîcheur est calculée sur la date de l'événement, pas sur celle de publication du fichier. Une source absente produit un rapport FAIL et un code de sortie d'erreur; le test automatisé corrige la source et relance le même pipeline.

Stockage et gouvernance

Le batch CSV est proportionné à un export ponctuel de cette taille. SQLite apporterait transactions et requêtes structurées, mais ajouterait des migrations et de la maintenance; streaming et entrepôt n'ont pas de besoin mesurable ici. Le champ name reste hors curated; le raw réel est exclu de Git et doit être supprimé ou anonymisé en fin de projet selon les conditions de la source. Les indicateurs décrivent des signalements, jamais un risque individuel.

Usage d'outils d'IA

J'ai utilisé GitHub Copilot comme aide pour certaines tâches d'implémentation, de documentation et de tests. Je reste responsable des choix initiaux et d'implémentation du projet, de la mise en place de la documentation, ainsi que de la compréhension et des choix présentés pour ce projet. Le pipeline et les tests ont été exécutés.

Consulter et relancer le projet

Le dépôt GitHub est public et constitue le livrable de code. Le lien ci-dessous est cliquable dans le PDF :

github.com/noahgog25/shark-attacks-collector

Installation et exécution

Je guide l'installation pas à pas dans le README à la racine du dépôt, avec des commandes pour macOS, Linux, PowerShell et cmd. J'y indique Python 3.11+, la création de l'environnement virtuel, l'installation depuis requirements.txt, puis les commandes de lancement et de tests. Depuis la racine du projet, je lance le pipeline avec `python -m src.pipeline`. Si je n'ai pas téléchargé l'export réel, le jeu synthétique permet de vérifier le fonctionnement.

Lire le code avec les documents du dépôt

Pour comprendre…	Ouvrir…
Le besoin et les indicateurs	docs/project_brief.md
Le flux de données	docs/architecture.md et docs/architecture_v1.svg / architecture_v2.svg
Les champs et règles	docs/data_dictionary.md et config/data_contract.yaml
Le choix de stockage	docs/ADR_001.md
Un échec et sa reprise	docs/incident_test.md et tests/test_collect.py
Les sources et leurs limites	docs/source_assessment.md et docs/rgpd_audit.md

Livrables et limites

Je produis un curated sans nom, des rejets avec motifs, un profil source, un rapport de run et des tableaux/visualisations. J'interprète les hotspots comme des comptes déclarés, non comme un taux de dangerosité. Mon rapport distingue le succès opérationnel PASS des alertes qualité; je garde l'export réel local et fournis un échantillon synthétique pour rendre l'exécution vérifiable.